

DSA STUDY BLUEPRINT SERIES

23. Valid Palindrome Remove Occurrences

Checking Valid Palindromes and String Modifications

Section 04 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Palindrome:** Reads same forward and backward, ignoring non-alphanumeric chars.
- **Remove Occurrences:** Deleting substrings recursively.

Memory & Execution Blueprint

r a c e c a r

[Left Ptr] →

⇐ [Right Ptr]

C++ Implementation Reference

Standard code layout and syntax

```
bool isPalindrome(string s) {  
    int l = 0, r = s.length() - 1;  
    while (l < r) {  
        while (l < r && !isalnum(s[l])) l++;  
        while (l < r && !isalnum(s[r])) r--;  
        if (tolower(s[l++]) != tolower(s[r--])) return false;  
    }  
    return true;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Palindrome Check	$O(N)$ time, $O(1)$ space
String deletion	$O(N^2)$ worst

Key Practices & Gotchas

- **Important:** Use two-pointer checks to avoid allocating string copies.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run two-pointer string validations or window updates for string "race" :

Step	Left Pointer Index	Right Pointer Index	Char Comparison	Outcome
1	left = 0 ('r')	right = 3 ('e')	'r' != 'e'	Validation stops (Not a palindrome)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Sliding Window Patterns:** Track character frequency maps within a moving boundary window size to find substring anagrams and permutations.
- **In-place reversals:** Swap characters inside word boundaries using double pointer offsets.
- **Related LeetCode Problems:** #3 Longest Substring Without Repeating Characters, #567 Permutation in String, #344 Reverse String.